Notes On a Reverse Engineering Algorithm

v0.1, 4 November 2026

1 The machine model

Choose a word size w and let $\mathbb{Z}_m = \mathbb{Z}/m\mathbb{Z}$. A register file is a finite set $\mathcal{R}$ of byte cells, while memory is a total function mapping addresses to bytes. The state space is

$$\Sigma \;=\; \mathbb{Z}_{2^8}^{\mathcal{R}} \times \mathbb{Z}_{2^8}^{\mathbb{Z}_{2^{64}}}.$$

A straight-line fragment P defines a function $[\![P]\!]$ from Σ into Σ, while an ABI defines an observation π from Σ into a finite set Y (for instance, the low byte of the return register together with final memory). The observable function for P is the composite $f_P = \pi \circ [\![P]\!]$.

All that follows is relative to this model. Faults, page permissions, timing, and concurrency are outside Σ, and none of the statements here addresses them.

2 Boxes

Definition 2.1. A *box* is a first-order formula $A(\sigma)$ in the theory of fixed-width bit-vectors and arrays, whose free variables range over Σ. Its extension is $\mathrm{Sat}(A) = \{\sigma \in \Sigma : \sigma \models A\}$.

A box is not a Cartesian product. For example, the formula $p = q$ over two pointer registers defines a box whose extension is the diagonal, not a rectangle. Boxes form a Boolean algebra under $\wedge$, $\vee$, $\neg$, and Sat is a homomorphism of Boolean algebras into the power set of Σ.

Proposition 2.2. *For boxes A and B,*

$$\mathrm{Sat}(A) \subseteq \mathrm{Sat}(B) \iff A \wedge \neg B \text{ is unsatisfiable}, \qquad \mathrm{Sat}(A) \cap \mathrm{Sat}(B) \neq \emptyset \iff A \wedge B \text{ is satisfiable}.$$

Proof. $\mathrm{Sat}(A) \setminus \mathrm{Sat}(B) = \mathrm{Sat}(A \wedge \neg B)$, and a set is empty exactly when its defining formula has no model. The second statement is $\mathrm{Sat}(A) \cap \mathrm{Sat}(B) = \mathrm{Sat}(A \wedge B)$. □

The solver returns either `sat`, `unsat` or `unknown`. An answer of `sat` includes a model, which provides a concrete witness state; `unsat` proves emptiness; `unknown` establishes nothing and is reported as such.

Example 2.3 (Aliasing)**.** Let P first store 7 at p, then 9 at q, and finally load the byte at p. When memory is represented by a single array M, the result is

$$\mathrm{read}(\mathrm{store}(\mathrm{store}(M, p, 7), q, 9), p) \;=\; \begin{cases} 9 & p = q, \\ 7 & p \neq q, \end{cases}$$

by the read-over-write axioms. Thus, for any allocation box B, the boxes $B \wedge (p = q)$ and $B \wedge (f_P = 9)$ have identical extensions. Proposition 2.2 establishes both inclusions using two unsatisfiability checks.

Proposition 2.4 (Projection of irrelevant inputs)**.** *Let $r \in \mathcal{R}$. Then f_P is independent of the cell r if and only if the formula*

$$f_P(\sigma) \neq f_P(\sigma[r := v])$$

in the free variables σ, v is unsatisfiable.

Proof. Independence of r means $f_P(\sigma) = f_P(\sigma')$ whenever σ and σ' agree off r. Every pair of this kind has the form $(\sigma, \sigma[r := v])$. The formula is satisfiable precisely when such a pair provides a counterexample. □

Only once this check succeeds is r removed from the input interface.

3 Execution fibers

Definition 3.1. Given f from X into Y and $y \in Y$, the *fiber* over y is $f^{-1}(y) = \{x \in X : f(x) = y\}$.

Example 3.2. Let $f(x) = 6x + 4$ on $\mathbb{Z}_{256}$. Then $f^{-1}(22) = \{3, 131\}$.

Proof. $6x + 4 \equiv 22 \pmod{256}$ is $6x \equiv 18 \pmod{256}$. Since $\gcd(6, 256) = 2$ divides 18, this is equivalent to $3x \equiv 9 \pmod{128}$. Because $3 \cdot 43 = 129 \equiv 1 \pmod{128}$, multiplying by 43 yields $x \equiv 387 \equiv 3 \pmod{128}$. The residues of $\mathbb{Z}_{256}$ that are congruent to 3 modulo 128 are 3 and 131. □

In general, $|f^{-1}(y)| \in \{0, \gcd(a, 2^w)\}$ for $f(x) = ax + c$ on $\mathbb{Z}_{2^w}$, so the observation by itself cannot distinguish the inputs when a is even.

Proposition 3.3 (Augmented observation)**.** *Let $x \in \{0, \ldots, 255\}$ and $\tilde{f}(x) = (y, k)$, where $6x+4 = y + 256k$ with $0 \leq y < 256$. Then $\tilde{f}$ is injective, and $x = (y + 256k - 4)/6$.*

Proof. The integer $6x + 4$ can be recovered from (y, k) as $y + 256k$, while x can be recovered from $6x + 4$ because multiplication by 6 is injective on $\mathbb{Z}$. □

For the fiber in Example 3.2: $6 \cdot 3 + 4 = 22$, $(22, 0)$ follows, while $6 \cdot 131 + 4 = 790 = 22 + 3 \cdot 256$ yields $(22, 3)$. The quotient k is additional observed state: the bits lost when truncating to one byte.

Remark 3.4 (Three coordinate systems). The same function can be represented in three ways: modular arithmetic on $\mathbb{Z}_{256}$; a Boolean circuit over $\mathbb{F}_2^8$ with explicit carry bits; and an embedding into $\mathbb{R}^8$ whose points are constrained to the hypercube vertices $\{0, 1\}^8$. Each model has the fiber $\{3, 131\}$ over 22. If the last model is relaxed to the solid cube, the fiber expands to include non-vertex points. Thus, the discreteness constraint belongs to the exact model and cannot be omitted.

4 Compiler fibers

Let S denote a set of source texts, θ a compiler profile (compiler, version, flags), and C_θ the map from S to byte strings obtained by compiling and extracting the relevant function or image.

Definition 4.1. For a target byte string b, the *compiler fiber* is $C_\theta^{-1}(b) \subseteq S$. A *partial right inverse* on D is a map I from D into S such that $C_\theta(I(b)) = b$ for every $b \in D$.

Strict source recovery means producing an element of $C_\theta^{-1}(b)$. We distinguish two notions:

$$\begin{aligned} \text{exact:} \quad & C_\theta(s) = b, \\ \text{semantic:} \quad & f_{C_\theta(s)} = f_b \text{ on the modelled domain.} \end{aligned}$$

Exact recovery implies semantic recovery. The converse does not hold, because different register allocations or instruction selections can compute the same function.

4.1 The affine family

Proposition 4.2. *Let $f(x, y) = ax + by + c$ on $\mathbb{Z}_{2^w}$. Then*

$$c = f(0,0), \qquad a = f(1,0) - c, \qquad b = f(0,1) - c,$$

and distinct triples $(a, b, c) \in \mathbb{Z}_{2^w}^3$ specify distinct functions.

Proof. Evaluate at the three basis points. The formulas recover (a, b, c) from f, so any two triples with the same f must coincide. □

Proposition 4.2 provides the inverse I. A candidate counts as exact only when recompiling $I(b)$ reproduces the function bytes; otherwise, it is reported as semantic.

Remark 4.3 (Non-uniqueness for polynomials). Uniqueness does not hold beyond degree one. For $w \geq 1$, the product $x(x-1)$ is always even, and therefore

$$2^{w-1}x(x-1) \equiv 0 \pmod{2^w} \quad \text{for all } x.$$

Thus, g and $g + 2^{w-1}(x^2 - x)$ define the same function on $\mathbb{Z}_{2^w}$ despite having different coefficients. Polynomial functions on $\mathbb{Z}_{2^w}$ form a proper quotient of the polynomial ring, so a recovered coefficient vector is merely a representative of its class.

4.2 Whole-program search

A source grammar has a finite set of holes $H = \{h_1, \ldots, h_n\}$, a finite choice set K_i with costs κ_i from K_i into $\mathbb{N}$ for each hole, and a finite set of profiles Θ with cost κ_0. An assignment selects one choice for each hole, and substitution is literal and applied at every occurrence. The candidate space is therefore

$$\Omega = \Theta \times K_1 \times \cdots \times K_n, \qquad \kappa(\theta, k_1, \ldots, k_n) = \kappa_0(\theta) + \sum_{i=1}^{n} \kappa_i(k_i).$$

Each $\omega \in \Omega$ specifies a complete project s_ω, and ω is accepted when $C_\theta(s_\omega) = b$ for every target.

Proposition 4.4. *Sort each coordinate by cost, then initialize a priority queue with the tuple of minimum choices. Repeatedly pop a minimum-cost tuple, emit it, and push each tuple formed by advancing one coordinate to its next choice, skipping any tuple already seen. This emits every element of Ω exactly once, in nondecreasing cost order.*

Proof. Advancing a coordinate cannot lower the cost, because the choices in each coordinate are sorted. Each tuple other than the initial one has a predecessor formed by moving back one coordinate, with cost no greater than its own. Induction on the sum of the coordinate indices shows that every tuple is pushed, since its predecessor is popped. The seen set prevents duplicates. Suppose a tuple u is popped after a tuple v with $\kappa(u) < \kappa(v)$. When v was popped, some ancestor u' of u along a chain of predecessors was in the queue, with $\kappa(u') \leq \kappa(u) < \kappa(v)$, contradicting the minimality of v. □

Corollary 4.5. *The first accepted candidate has the minimum declared cost among all accepted candidates in Ω. If the budget expires after N candidates, then every rejected candidate whose cost is below the last emitted cost has been tested. No conclusion can be drawn about Ω beyond those N, nor about any sources outside Ω.*

The size $|\Omega| = |\Theta| \prod_i |K_i|$ grows exponentially in n. Ordering makes the search useful, but does not make it polynomial.

5 Color refinement

Let $G = (V, E, \ell)$ be a directed multigraph with vertex labels ℓ and edge slots $s(e)$ recording operand position; operands of commutative operations share a slot.

Definition 5.1. Set $\chi_0 = \ell$ and

$$\chi_{t+1}(v) = \Big(\chi_t(v),\ \{\!\{(s(e), \chi_t(u)) : e = (u, v)\}\!\},\ \{\!\{(s(e), \chi_t(u)) : e = (v, u)\}\!\}\Big),$$

where $\{\!\{\cdot\}\!\}$ is a multiset. Let $\mathcal{P}_t$ denote the partition of V into classes of χ_t.

Lemma 5.2. *$\mathcal{P}_{t+1}$ refines $\mathcal{P}_t$. If $\mathcal{P}_{t+1} = \mathcal{P}_t$, then $\mathcal{P}_{t+k} = \mathcal{P}_t$ for all k. Stabilization takes at most $|V|$ rounds.*

Proof. $\chi_{t+1}(v)$ includes $\chi_t(v)$ as its first component, so identical new colors entail identical old colors. If two rounds produce the same partition, then χ_{t+1} is a bijective relabelling of χ_t on the classes, and the next round encounters the same multisets up to that relabelling. Each strict refinement increases the number of classes, which is bounded by $|V|$. □

Colors are compared as exact tuples, preventing hash collisions from merging distinct classes.

Proposition 5.3. *If $G \cong G'$ under a label- and slot-preserving isomorphism, the stable color histograms of G and G', refined jointly, are equal. Consequently, different histograms prove non-isomorphism.*

Proof. For an isomorphism φ, $\chi_t(\varphi(v)) = \chi_t(v)$ holds for all t by induction on t, since it maps the in- and out-neighbourhoods bijectively while preserving slots. □

Example 5.4 (The converse fails)**.** Let C_6 be the six-cycle and $2K_3$ be two disjoint triangles, both with uniform labels. Each graph is 2-regular and has six vertices, so every vertex receives the same color in every round and the histograms match. Nevertheless, the graphs are not isomorphic.

Betti numbers of the underlying 1-complex distinguish them: β_0 counts the components, and $\beta_1 = |E| - |V| + \beta_0$ is the cycle rank:

$$(\beta_0, \beta_1)(C_6) = (1, 1), \qquad (\beta_0, \beta_1)(2K_3) = (2, 2).$$

Remark 5.5. Structural matching and semantic equality are independent properties. On $\mathbb{Z}_{2^w}$, $x \cdot 3$ and $(x \ll 1) + x$ represent equal functions despite having different graphs. Conversely, $x + y$ and $x \oplus y$ share the same untyped topology but differ at $x = y = 1$.

6 Counterexample-guided synthesis

Given a finite parameter domain $\mathcal{P}$, a finite state domain Σ_0, and a violation formula $V(p, \sigma)$, the goal is

$$\exists p \in \mathcal{P} \;\; \forall \sigma \in \Sigma_0 : \neg V(p, \sigma).$$

The procedure maintains a finite set $T \subseteq \Sigma_0$ of counterexamples. It searches for p satisfying $\bigwedge_{\tau \in T} \neg V(p, \tau)$; if none exists, the grammar is exhausted. Otherwise, it searches for σ satisfying $V(p, \sigma)$. If none exists, it accepts p; if σ exists, it adds σ to T.

Proposition 6.1. *Accepted parameters are correct, and the procedure halts within at most $|\mathcal{P}|$ iterations.*

Proof. Acceptance is equivalent to the unsatisfiability of $V(p, \sigma)$ over the whole of Σ_0. If σ is added for p, then $V(p, \sigma)$ holds, which means p violates the new constraint and cannot be proposed again. Each iteration eliminates at least one parameter. □

Exhaustion establishes only that no parameter in this grammar works. Solver unknown is reported as unresolved.

7 Loop contracts

Let a loop have a one-lap transition $T(\sigma, \sigma')$, extracted from the machine code, a guard g, an initial condition Init, an invariant candidate J, a postcondition Q, and a ranking function ρ into $\mathbb{N}$.

Theorem 7.1. *Suppose the following formulas hold:*

$$\begin{aligned}
&\mathrm{Init}(\sigma) \Rightarrow J(\sigma),\\
&J(\sigma) \wedge g(\sigma) \wedge T(\sigma, \sigma') \Rightarrow J(\sigma'),\\
&J(\sigma) \wedge \neg g(\sigma) \Rightarrow Q(\sigma),\\
&J(\sigma) \wedge g(\sigma) \wedge T(\sigma, \sigma') \Rightarrow \rho(\sigma') < \rho(\sigma).
\end{aligned}$$

Then every execution starting from Init *terminates in a state satisfying Q.*

Proof. Induction on the number of laps shows that J holds at every loop head. During any run, ρ decreases strictly in $\mathbb{N}$ at each lap, so the run has at most $\rho(\sigma_0)$ laps. At exit, $\neg g$ holds along with J, and therefore Q. □

Each implication is verified by checking that its negation is unsatisfiable. For a counter i that runs to a bound n, $\rho = n - i$. A constant ρ fails the fourth condition, producing a counterexample.

Remark 7.2. An XOR accumulator $a' = a \oplus k$ is not affine over $\mathbb{Z}_{2^w}$, but it is linear over $\mathbb{F}_2^w$. Thus, the invariant language depends on the chosen coordinates, as noted in the remark on three coordinate systems.

8 Prototype refinement as a fixed point

Let F denote the set of recovered functions. For each $g \in F$, let $\delta(g)$ be the prototype inferred by the decompiler from the body of g: the model, arity, and a storage triple $(\mathrm{space}, \mathrm{offset}, \mathrm{size})$ with a type for each parameter and the return. For a direct call site c in a caller h with target g, let $\gamma(c)$ denote the prototype inferred from the h side of the call.

Definition 8.1. The evidence is *consistent* at c if $\gamma(c)$ and $\delta(g)$ match in model, variadicity, custom storage, arity, every storage triple, and every type. A disagreement is a *conflict*.

Conflicts indicate disagreements between independent inferences; they do not prove anything about the true ABI.

A call is *trusted* if its caller has no unresolved relocation reads, bad data, or unimplemented instructions. For a caller h, define the manifest

$$m(h) = \{(g, \delta(g)) : h \text{ calls } g \text{ directly},\ g \neq h,\ \delta(g) \text{ supported}\},$$

where supported means non-variadic, using standard register or stack storage and supported types. Let Φ denote the operator that re-decompiles every caller h using the hypotheses $m(h)$ and recomputes δ and γ.

The decompiler checks each hypothesis against the storage it reconstructs. If it rejects any hypothesis, or the result contains unresolved machine operations, the previous result for h is retained.

Proposition 8.2. *Let $\eta(h)$ be a hash of $m(h)$ combined with the backend identity, and record the last adopted value $\eta^*(h)$. If iteration stops because $\eta^*(h) = \eta(h)$ holds for every caller h, the current state is a fixed point of Φ in this sense: rerunning Φ would provide each caller with exactly the hypotheses used in its last decompilation.*

Proof. For a deterministic backend, decompiling h depends only on h, the backend, and $m(h)$. Assuming there are no hash collisions, equal hashes imply equal manifests, so Φ reproduces the current evidence. □

No lattice order on prototypes is claimed under which Φ is monotone, so convergence is not generally guaranteed. Termination is enforced by a round limit R, a time budget, and a rule permitting each pair (h, η) to be attempted at most once. The outcome is reported as one of: fixed point, unresolved (a round with no adoption), or budget exhausted.

Example 8.3. Let $\texttt{leaf}(x, y) = x + y$ take x in EDI and y in ESI, and suppose the caller sets only EDI = 7 before the call. In its first decompilation, the caller sees one argument. With $m(\texttt{caller}) = \{(\texttt{leaf}, \delta(\texttt{leaf}))\}$, the call becomes binary, and the forwarded ESI appears as a caller parameter. A hypothesis assigning the first argument to the wrong register is rejected because of the storage mismatch. On rerun, $\eta^* = \eta$ and no reanalysis occurs; corrupting η^* triggers exactly one reanalysis, which restores the same η.

9 Limitations

All results in these notes are proved with respect to the machine model Σ and a supplied, rather than inferred, ABI. Accordingly, box containment, the fibers of Section 3 and the loop invariants of Section 7 make no claims about faults, page permissions, concurrency or timing. They hold only if the binary actually uses the stated calling convention.

Recovery is judged by the strict standard $C_\theta(s) = b$. A candidate that computes the same function over the modelled domain is classified as semantic agreement, never as recovered source, since different register allocations and instruction selections routinely implement the same function. The reverse gap is equally important: if the search over Ω runs out of budget, the sole conclusion is that the candidates tested failed. This does not show that no source exists, either within the grammar or outside it.

The structural tools have the same one-sided character. Different stable 1-WL histograms prove that two graphs are not isomorphic, whereas matching histograms prove nothing, as C_6 and $2K_3$ show; moreover, graph isomorphism neither implies nor rules out semantic equality. Finally, a prototype fixed point certifies only that inference has stopped changing. Since no monotone order is known for Φ, convergence is enforced through budgets rather than guaranteed, and a stable set of prototypes does not prove that the recovered program is equivalent to the original.